LazyMem：广泛检索、选择性构建，实现高效的智能体长期记忆

Jing Yu[1], Yibo Zhao[1], Jiaming Zhang[1], Xiang Li[1]*
[1]华东师范大学数据科学与工程学院

摘要

长期记忆使 LLM 智能体能够利用过去的交互，但对话历史很快就会超出上下文窗口，迫使智能体在查询时检索相关子集。由于有用证据稀疏地散落在冗长的对话中，检索面临一个根本矛盾：扩大召回范围可以提高覆盖率，却会使下游推理充斥噪声；而在写入时压缩记忆虽能降低检索难度，却会不可逆地丢弃未来查询可能需要的细节。我们提出 LazyMem，通过将所有记忆构建推迟到查询时来化解这一矛盾。给定检索得到的候选池，一个轻量级模型在相互重叠的并行窗口中对其进行处理，仅选择性地保留并压缩与查询相关的内容。该模型先经过监督微调，再通过强化学习进行训练；其奖励同时鼓励模型识别相关消息，并生成忠实于来源且有助于回答查询的压缩结果。在 LongMemEval 上，LazyMem-4B 的 LLM 裁判准确率达到 0.85，超过最强的非预言机基线，同时答案上下文中仅使用 213 个记忆词元，比该基线少 21.0×。它还能够在不进行目标领域训练的情况下泛化到 LoCoMo，并相较此前的查询时基线降低平均延迟。代码见 https://github.com/allacnobug/LazyMem。

1 引言

通过维护长期对话记忆，LLM 智能体可以保存过去与用户的交流，并在后续对话中加以利用 (Park 等, 2023; Packer 等, 2024; Zhong 等, 2024)。然而，累积的对话历史可能远远超出上下文窗口，而不加选择地全部纳入会使相关证据淹没在噪声中。因此，智能体必须从历史中检索与查询相关的子集，并将其纳入当前上下文。这一检索步骤在目的上与 RAG 相似，但底层语料的质量和结构有所不同。传统 RAG 假设使用经过整理、信息密集的段落 (Lewis 等, 2020; Karpukhin 等, 2020)；相比之下，原始智能体记忆结构松散且内容冗长，关键信息散落在各个交互轮次中 (Maharana 等, 2024; Wu 等, 2025)。这种不匹配造成了两难困境。噪声和碎片化使与查询相关的证据难以检索；然而，为提高证据覆盖率而扩大召回池，又会增加上下文负担并进一步稀释关键信息 (Shi 等, 2023; Liu 等, 2024)。

大多数现有方法试图通过先构建后检索范式来缓解这一困境。它们在存储之前，对原始交互进行总结 (Zhong 等, 2024; Fang 等, 2026)、重写 (Chhikara 等, 2025; Zhang 等, 2026b)、聚合 (Sun 和 Zeng, 2025; Li 等, 2026)，或将其连接成结构化记忆单元 (Xu 等, 2026; 2025)，并在查询时检索相关单元。通过预先压缩和组织原始交互，这一范式有助于查询匹配，并减轻下游推理负担。然而，由于这一记忆构建过程必然与查询无关，它可能会丢弃后来被证明相关的细节。AMA-Bench (Zhao 等, 2026) 报告的一项消融研究表明，即使排除了检索错误，输入构建后记忆的模型也明显不如输入相应黄金交互轮次的模型；这揭示了记忆构建过程中引入的信息损失（详见附录 A）。

一种自然的替代方案是保留原始交互，并将构建推迟到查询时。有了查询信息，系统便可以确定应保留哪些事实以及应以多大力度压缩它们，从而避免查询无关构建所固有的永久信息损失。Wu 等 (2026) 提出的查询驱动剪枝（Query-Driven Pruning）提供了一个起点：系统检索原始对话，再由一个独立的语言模型通过提示抽取与查询相关的内容，随后将所得证据交给答案模型。这一设计证明了先检索后构建范式的可行性：保留原始交互，并以查询为条件构建证据。然而，该方法存在两项局限。第一，尽管抽取与答案生成相互解耦，检索覆盖率和上下文负担之间的权衡依然存在。抽取模型仍然一次性处理整个召回池；随着冗长且含噪的上下文稀释关键信息，它仍容易受到上下文腐化的影响。第二，基于提示的抽取还在抽取质量、成本与延迟之间引入了额外权衡。大型模型无需特定任务训练即可获得尚可的

*通讯作者：xiangli@dase.ecnu.edu.cn

抽取质量，但成本高、速度慢；轻量级模型则成本更低、速度更快，却会在缺乏专门训练时表现不佳。这些发现引出了本工作的核心问题：在实际的成本和延迟约束下，智能体记忆系统应如何平衡信息保真度与上下文负担?

本文提出 LazyMem，一种旨在应对这些权衡的先检索后构建智能体记忆系统。其名称体现了两层“惰性”：一方面，它在写入时不进行任何有损构建，而是将记忆构建推迟到查询到来之时；另一方面，它将构建工作交给轻量级记忆处理模型，从而使主推理模型无需处理原始记忆。在查询时，LazyMem 首先检索一个较大的原始观测池，以确保充分的证据覆盖；随后，记忆处理模型为下游推理构建紧凑的查询条件化证据。为控制上下文负担，它在相互重叠的窗口中处理召回池，使每个输入都足够短，以缓解上下文腐化，同时保留跨窗口边界的证据。尽管模型规模较小，为实现高质量构建，我们仍使用强化信号进行训练，共同优化其对原始交互的保真度和对下游推理的效用。各窗口并行处理，使延迟在召回池增长时仍得到控制；同时，较小的模型规模使推理成本保持较低水平。

我们的主要贡献概括如下。

- 我们提出 LazyMem，一种先检索后构建的智能体记忆系统。它保留原始交互，在查询时进行广泛检索，并通过一个在相互重叠的并行窗口上运行的轻量级模型构建紧凑证据，从而在限制上下文长度和延迟的同时，避免写入时不可逆的信息损失。
- 我们开发了一个采用 SFT 和格式门控强化学习训练的 4B 记忆处理模型，并使用联合奖励来提升选择准确性、忠实度和查询效用。消融实验表明，SFT 改善了格式遵循能力，RL 则进一步提升了记忆构建质量；二者结合的效果优于单独使用提示。
- 在两个公认基准上，LazyMem-4B 在 LongMemEval 上仅使用 213 个答案上下文记忆词元，便取得 0.85 的 LLM 裁判准确率；其词元数量比纯检索方法少 68.7×，比最强非预言机基线少 21.0×。在不进行目标领域训练的情况下，它在 LoCoMo 上达到 0.68，同时相较此前的查询时方法降低了平均端到端延迟。

2 相关工作

MemoryBank (Zhong 等, 2024) 和 MemGPT (Packer 等, 2024) 等早期工作为 LLM 智能体配备了超越有限上下文窗口的持久记忆。尽管设计各不相同，这些系统都采用一条通用流程：存储过去的交互，检索与当前查询相关的信息，并将检索内容纳入模型上下文。从过去交互何时被转化为任务可用记忆的角度出发，我们将现有方法大致归纳为两种范式：先构建后检索和先检索后构建。

先构建后检索方法在未来查询尚未确定之前，便将交互转化为紧凑或结构化的记忆表示。其中一类方法通过显式操作管理持久记忆，包括基于提示执行一组预定义的 ADD/UPDATE/DELETE 操作 (Chhikara 等, 2025)、通过强化学习习得操作策略 (Yan 等, 2026; Yue 等, 2026)，以及构建可复用、可演化的记忆技能 (Zhang 等, 2026b)。另一类方法则以层次化方式组织交互历史，将具体交互整合为具有不同粒度和抽象层级的时间、主题及关系结构 (Fang 等, 2026; Li 等, 2026; Xu 等, 2026)。在检索侧，研究者针对这些表示设计了相应机制，包括原子笔记之间的语义连接 (Xu 等, 2025)、实体–关系图 (Chhikara 等, 2025)、层次化自顶向下搜索 (Sun 和 Zeng, 2025)，以及通过搜索得到的模块化记忆架构 (Zhang 等, 2026a)。尽管这些方法卓有成效，但其写入时构建必然与查询无关，因此可能与不可预见的未来信息需求产生错配。

先检索后构建方法则保留接近原始交互的细粒度记录，将建模推迟到构建阶段：查询确定后，它通过筛选、压缩和重组召回内容，形成特定于任务的上下文。尽管重排序 (Yu 等, 2024)、上下文压缩 (Xu 等, 2024) 和查询聚焦总结 (Edge 等, 2024) 等相关思想已在传统 RAG 中得到充分研究，但这一范式在智能体记忆领域仍缺乏探索。NanoMemory (Wu 等, 2026) 保留原始对话，并将交互轮次隔离检索与查询驱动剪枝相结合，将召回的会话提炼为紧凑、信息密集的上下文。然而，由于它一次性处理整个召回池，上下文腐化在记忆构建期间仍是一项挑战。此外，其基于提示的记忆处理模型还面临构建质量与推理效率之间的权衡。

我们的 LazyMem 以这一范式为基础，将查询条件化记忆构建表述为由轻量级模型实现的习得策略。它通过相互重叠的窗口并行处理大型候选池，在限制每个构建步骤所处理上下文的同时，控制查询时延迟。

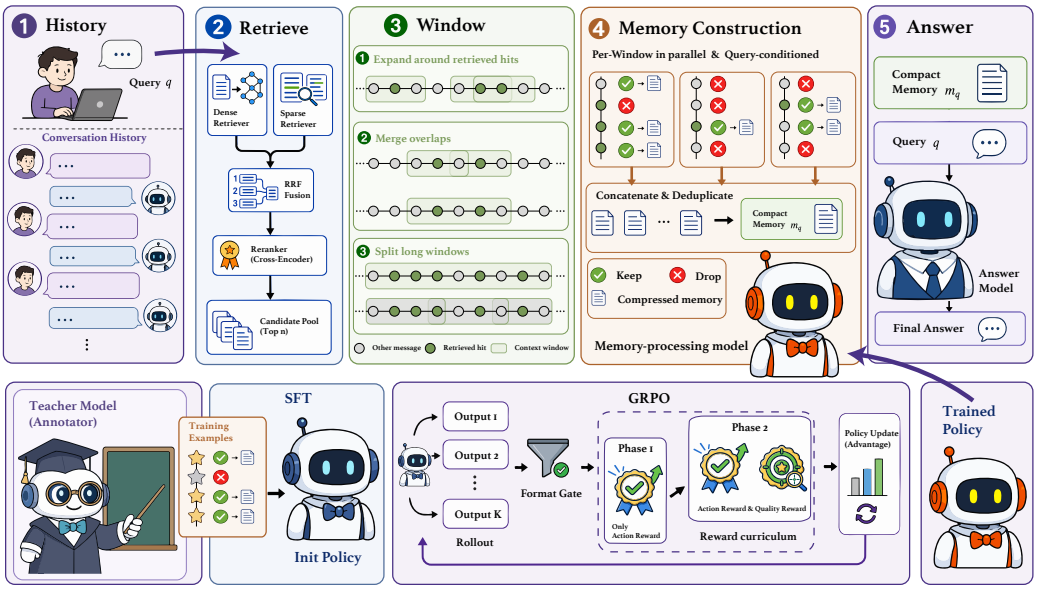


图 1: LazyMem 概览。在查询时（上部），混合检索、上下文窗口化与选择性构建将冗长的对话历史转化为供回答模型使用的紧凑记忆。记忆处理模型首先通过教师监督微调进行初始化，随后使用包含动作奖励与质量奖励的强化学习进一步优化（下部）。

3 方法

3.1 LazyMem 流水线

我们考虑在冗长交互历史上回答查询 q 的问题，其中智能体必须利用散布于过往用户消息与助手消息中的相关信息。LazyMem 将其处理为一个惰性的先检索、后构建记忆系统（见图 1）：写入时原样存储消息而不进行处理，将所有工作推迟到查询时，此时系统按三个阶段运行。

广泛检索。 LazyMem 首先使用标准混合检索构建一个至多包含 n 条候选消息的高召回候选池 $\mathcal{R}_q$。稠密（双编码器）检索器与稀疏（BM25）检索器针对 q 并行运行，并通过倒数排序融合 (Cormack 等, 2009) 合并各自的排序列表；随后，交叉编码器重排序器对融合后的候选项打分，并保留前 n 个。所有检索组件均为现成组件。LazyMem 的贡献在于如何对所得候选池进行后续处理。

选择性构建。 随后，构建模块将 $\mathcal{R}_q$ 压缩为紧凑的查询条件化记忆上下文 m_q。该模块将候选池划分为相互重叠的证据窗口，并将一个经强化学习训练的轻量级记忆处理模型 π_θ 并行应用于所有窗口（见第 3.2 节）。窗口化在保留跨边界证据的同时，限制每次模型调用所处理的消息数；随着候选池增大，并行机制则可限制时延。第 3.3 节说明了 π_θ 如何在模型规模较小的情况下仍实现高质量的记忆构建。

作答。 最后，回答模型接收查询 q 与紧凑上下文 m_q，并据此生成预测结果。

3.2 查询条件化记忆构建

构建过程分为两步：首先通过历史窗口化恢复每条检索消息周围的局部对话上下文，随后由记忆处理模型将每个窗口压缩为与查询相关的内容。

历史窗口化。 $\mathcal{R}_q$ 中的消息脱离了其周围对话，因此孤立处理这些消息可能破坏对话连贯性。例如，要解析一条检索消息中的代词，可能需要参考此前的对话轮次。因此，LazyMem 直接从交互历史中重建局部上下文。令 $d_1, \ldots, d_{|\mathcal{R}_q|}$ 表示按时间顺序排列的检索消息，并令

idx(·) 返回消息在历史中的位置。将每个 d_i 扩展为一个跨度，该跨度包含其自身以及前后各 w 条消息。这些相邻消息提供理解所需的上下文，并且通常能够找回检索所遗漏的相关信息。只要两条相邻检索消息的跨度发生重叠，就将它们合并为一个连续窗口：

$$\mathrm{idx}(d_{i+1}) - \mathrm{idx}(d_i) \le 2w. \tag{1}$$

传递地应用该规则会得到 M 个不含重复上下文的极大窗口。这里，窗口长度以消息数量计量。超过最大长度 L 的窗口会被拆分为相互重叠的子窗口，从而限制每次模型调用所处理的消息数量。我们将最终的窗口集合记为 $\{W_1, \ldots, W_P\}$，其中 $P \ge M$，并在消融实验（第 4.4 节）中评估 w 的影响。

记忆处理模型。 模型 π_θ 以单条消息为粒度，独立处理每个 (q, W_j) 对。对于每条消息 $x \in W_j$，模型预测一个动作 $\alpha_x \in \{\mathrm{Keep}, \mathrm{Drop}\}$：被保留的消息会被重写为仅保留查询相关内容的压缩形式 $\tilde{x}$，而被丢弃的消息不会产生任何输出。当相互重叠的子窗口对同一条消息作出预测时，我们依据 idx(·) 进行去重，优先采用 Keep，并按窗口顺序打破平局。随后，将保留下来的压缩结果按时间顺序拼接成 m_q，并与 q 一同传递给回答模型。

3.3 记忆处理模型的训练

我们首先对 π_θ 进行监督微调（SFT），随后采用基于组的强化学习（RL）进行训练。

监督微调。 我们利用源交互构建 SFT 样例，其中的消息带有金标和非金标标签。源基准将金标消息标注为回答所必需的支撑证据；非金标消息则未经标注，但不一定不相关，仍可能提供辅助上下文。利用上述候选检索与历史窗口化方法，我们将查询和每个证据窗口提供给一个遵循指令的教师模型，由其执行记忆处理任务：模型将每条消息标注为 Keep 或 Drop，并为每条 Keep 消息生成与查询相关的压缩文本。若教师模型丢弃金标消息、产生无法解析的输出、违反模式约束，或返回的决策数量与窗口大小不匹配，我们便丢弃该实例。每个保留的样例都将一个查询和一个证据窗口映射为逐消息动作以及所有 Keep 消息的压缩结果。在这些样例上进行监督微调，可以让模型学会输出格式及基本的 Keep/Drop 行为；所得检查点将作为参考策略 π_{ref}。完整实现细节见第 4.1 节。

基于组的策略优化。 我们以 π_{ref} 为起点，使用经过如下修改的组相对策略优化（GRPO；(Shao 等, 2024)）进一步优化 π_θ。每个提示 p 都将查询 q 与一个证据窗口配对。采样策略 $\pi_{\theta_{\mathrm{old}}}$ 生成 G 个候选输出 $\{o^k\}_{k=1}^{G}$，每个输出都会从式 6 的复合奖励中获得一个标量奖励 $r^k = R(o^k; p)$。每个输出的优势均相对于其所在组计算：

$$\hat{A}^k = \frac{r^k - \mathrm{mean}\{r^l\}_{l=1}^{G}}{\mathrm{std}\{r^l\}_{l=1}^{G} + \delta}, \tag{2}$$

其中，δ 是用于保证数值稳定性的小常数。对于每个生成的词元 o_t^k，我们计算当前策略与采样策略之间的重要性采样比率：

$$\rho_t^k = \frac{\pi_\theta(o_t^k \mid p,\ o_{<t}^k)}{\pi_{\theta_{\mathrm{old}}}(o_t^k \mid p,\ o_{<t}^k)}. \tag{3}$$

我们采用标准的对称 PPO 裁剪，裁剪比率为 ϵ，从而得到逐词元代理目标：

$$l_t^k = \min\!\big(\rho_t^k\,\hat{A}^k,\ \mathrm{clip}(\rho_t^k,\ 1{-}\epsilon,\ 1{+}\epsilon)\,\hat{A}^k\big). \tag{4}$$

我们保留了 GRPO 中较小的参考策略 KL 惩罚，使策略维持在 π_{ref} 附近，这有助于保持 SFT 阶段学到的输出格式。由此得到的目标为：

$$\mathcal{J}(\theta) = \mathbb{E}_{p,\ \{o^k\}_{k=1}^{G} \sim \pi_{\theta_{\mathrm{old}}}} \left[\frac{\sum_{k=1}^{G} \sum_{t=1}^{|o^k|} l_t^k}{\sum_{k=1}^{G} |o^k|} - \beta\, D_{\mathrm{KL}}\big(\pi_\theta \,\|\, \pi_{\mathrm{ref}}\big) \right], \tag{5}$$

我们关于 θ 最大化该目标。遵循 DAPO (Yu 等, 2025)，我们使用 $\sum_k |o^k|$ 在组内所有词元上对代理目标进行归一化，而不是对每次采样分别取平均。这样可以避免降低较长采样的权重。按照式 4，裁剪仍采用对称形式。KL 项 D_{KL} 使用低方差 k_3 估计器 (Shao 等, 2024)，其强度由 β 控制。

奖励设计。 复合奖励结合了动作奖励 R_{act} 与质量奖励 R_{qual}；二者均被归一化至 $[0,1]$，并由格式有效性进行门控：

$$R(o;\, p) = \mathbb{I}[o \in \mathcal{V}]\,\big((1-\lambda_{\text{qual}})R_{\text{act}} + \lambda_{\text{qual}} R_{\text{qual}}\big)\,. \tag{6}$$

若输出是一个符合规定模式的单一 JSON 数组、恰好为窗口中的每条消息包含一个决策，并为每条 Keep 消息提供所需的压缩内容，则该输出有效（$o \in \mathcal{V}$）；任何违规情况都会得到零奖励。混合权重 $\lambda_{\text{qual}} \in [0,1]$ 及其调度方式将在下文的奖励课程中说明。

动作奖励。 我们复用源数据集自身的标注：被标为查询支撑证据的消息属于金标消息，而按照上述约定，所有未标注消息均属于非金标消息。令 A_{g} 为被赋予 Keep 动作的金标消息数除以金标消息总数，令 A_{n} 为被赋予 Drop 动作的非金标消息数除以非金标消息总数。对于同时包含这两类消息的窗口，动作奖励为：

$$R_{\text{act}} = \eta A_{\text{g}} + (1-\eta) A_{\text{n}}, \tag{7}$$

其中，$\eta > 0.5$ 为金标消息的保留赋予了更高权重。我们分别计算每个类别内的准确率，而不是在所有消息上统一计算，以免大量简单的 Drop 决策掩盖少量关键的 Keep 决策。采用非对称权重是因为：丢弃必需证据对于下游回答模型而言不可逆，而保留额外上下文虽然会引入上下文负担，却仍保存了证据。如果窗口只包含一个类别，则直接使用相应类别的准确率。η 的训练取值及相关诊断结果见附录 E.1。

质量奖励。 质量奖励对保留集合 $\mathcal{K} = \{x : \alpha_x = \text{Keep}\}$ 进行评分。它仅评判保留内容的质量，而保留是否为正确动作则由 R_{act} 评判。一个 LLM 评审器从两个维度对每个 $x \in \mathcal{K}$ 评分，两个维度均以 $\{0,1,2\}$ 分别表示不满足、部分满足和完全满足：忠实度得分 f_x 衡量压缩内容是否得到源消息的支持，效用得分 u_x 则衡量该内容对于回答查询的有用程度。我们还通过专家盲标研究进一步验证奖励评审器的可靠性，其方案与结果见附录 E.2。逐消息质量得分为：

$$Q(x) = \mathbb{I}[f_x > 0]\,\frac{f_x + u_x}{4} \;\in\; [0,1]. \tag{8}$$

该指示函数以忠实度作为门控条件：完全不忠实的内容（$f_x = 0$）未得到源消息支持或与其冲突，因此无论表面效用如何都获得零奖励。部分忠实的内容（$f_x = 1$）尽管不够精确，但仍以源消息为依据，因而能够通过门控，不过其得分低于完全忠实的内容（$f_x = 2$）。通过门控后，忠实度与效用以加法形式共同影响得分，因此任一维度有所欠缺都会逐步降低得分，而不会触发严格的通过/失败边界。与二元门控相比，这种分级信号能够为 GRPO 的优势归一化（式 2）提供区分度更丰富的采样结果。在保留消息上对 $Q(x)$ 取平均，即可得到窗口级奖励：

$$R_{\text{qual}} = \begin{cases} 0, & \mathcal{K} = \emptyset, \\ \dfrac{1}{|\mathcal{K}|}\displaystyle\sum_{x \in \mathcal{K}} Q(x), & \text{otherwise.} \end{cases} \tag{9}$$

奖励课程。 混合权重 $\lambda_{\text{qual}} \in [0,1]$ 将复合奖励维持在 $[0,1]$ 范围内。我们分两个阶段对其进行调度。在阶段 1 中，$\lambda_{\text{qual}} = 0$，因此训练仅依赖格式门控和成本较低、基于规则的动作奖励。这样可以先让策略学会作出正确的 Keep/Drop 决策，而不会产生 LLM 评审成本。一旦验证集上的格式有效率超过阈值 τ，训练便进入阶段 2；此时，固定的 $0 < \lambda_{\text{qual}} < 1$ 会激活质量奖励，同时在剩余训练过程中继续保留动作奖励。将质量奖励推迟到策略已经能够生成格式规范的输出之后，可以避免在训练早期将评审调用浪费于无法解析的采样输出。评分标准与评审器配置见附录 E.2，奖励混合方式与课程设置见附录 E.3，完整评审提示见附录 G.2 （代码清单 2）。

4 实验

我们的实验围绕本文研究的核心权衡展开：在实际成本与时延约束下，既保留有用的记忆信息，又控制上下文开销。我们围绕以下四个研究问题组织评估。

RQ1（有效性）：无论在域内还是域外，LazyMem 是否都优于有竞争力的记忆基线（第 4.2 节）？

RQ2（效率）：以推理时延和回答上下文记忆词元数衡量时，这些性能提升是否以更低的上下文开销实现（第 4.3 节）？

RQ3（消融）：历史窗口半径以及逐步叠加的 SFT 和 RL 阶段如何影响性能（第 4.4 节）?

RQ4（进一步分析）：LazyMem 会在哪些情形下失败，错误又如何分布在流水线的各个阶段（第 4.5 节）?

完整的效率测量结果、实现细节、错误审计结果和伦理考量分别见附录 C、D、F 和 6。

4.1 实验设置

数据集。 我们在两个长上下文记忆基准上进行评估。**LongMemEval** (Wu 等, 2025) 包含 500 个问题，每个问题均以一段跨多个会话的长篇用户–助手历史为依据。由于我们的方法需要训练数据，因此按问题类型进行分层采样，将该数据集划分为 360 个训练样本、40 个验证样本和 100 个测试样本。所有基线均使用相同的回答模型在我们的测试划分上重新评估，因而所报告的数值彼此之间可直接比较，但不能与先前公布的全量数据集结果直接比较。**LoCoMo** (Maharana 等, 2024) 严格仅用于测试，既不用于 SFT，也不用于 RL。遵循 Zhang 等 (2026b)，我们评估最后两段对话中的第 1–4 类问题（共 314 个问题）。因此，LongMemEval 衡量域内性能，而 LoCoMo 衡量域外泛化能力。

训练。 我们使用 Qwen3-4B (Yang 等, 2025) 作为基础记忆处理模型，首先应用监督微调，使其学会记忆选择与压缩的格式。SFT 数据来自 LongMemEval 训练划分中按问题类型分层抽取的 100 个问题，并由 DeepSeek-V4-Flash (DeepSeek-AI 等, 2026) 进行标注。由于窗口中的大多数消息没有金标准证据标注，教师模型的输出以 Drop 为主。因此，我们在窗口层面进行均衡，以缓解这种动作不平衡，最终得到 776 个 SFT 样本。

随后，我们使用 GRPO (Shao 等, 2024) 进行训练，并采用对称 PPO 截断和 DAPO 风格的组级词元归一化 (Yu 等, 2025)。我们根据提示中是否包含金标准证据，按 50/50 的比例均衡 RL 提示，确保包含金标准证据的窗口得到充分表示。按照第 3.3 节的两阶段课程，奖励在式 6 的格式门控下，结合基于规则的动作奖励 R_{act} 与由 LLM 评判的质量奖励 R_{qual}。

基线。 我们将 LazyMem 与覆盖主要记忆构建范式的方法进行比较。(i) 预言机参考：*Oracle Turn* 和 *Oracle Session* 将金标准证据所在的轮次或会话直接提供给回答模型，作为参考点。(ii) 仅检索而不构建：*RAG Top20/Top50* 使用与 LazyMem 相同的混合检索（第 3.1 节），随后按时间顺序拼接排名靠前的结果，不再进行进一步处理。(iii) 先构建后检索：在查询到来前构建或更新记忆的系统。此类方法包括无需训练的 *LightMem* (Fang 等, 2026)、*StructMem* (Xu 等, 2026)、*Mem0* (Chhikara 等, 2025) 和 *MemoryBank* (Zhong 等, 2024)，以及经过训练的 *MemSkill* (Zhang 等, 2026b) 和 *MemT* (Yue 等, 2026)。(iv) 先检索后构建：*NanoMemory* (Wu 等, 2026)，它保存原始消息，并在推理时构建与查询相关的上下文，与 LazyMem 属于同一范式。除非另有说明，基线使用启用思考模式的 Qwen3-32B (Yang 等, 2025) 作为记忆处理模型，为各系统构建记忆；对于 LazyMem，我们同时报告未经任务特定训练、仅使用提示的 Qwen3-32B 变体，以及经过训练的 LazyMem-4B 模型。

检索、窗口化与回答模型。 对于 LazyMem，我们使用第 3.1 节所述的混合检索流水线，检索由 n=50 条消息构成的候选池，其中使用 Qwen3-Embedding-8B (Zhang 等, 2025) 生成稠密嵌入，使用 Okapi BM25[1] 进行稀疏检索，使用 BGE-Reranker-v2-M3[2] 进行重排序，并以半径为 w=2 的上下文窗口扩展每条消息。在 SFT 数据构建、RL 提示和推理中，我们均将每个子窗口限制为至多 L=8 条消息，并采用 s=7 的步长，因此相邻的完整子窗口会重叠一条消息；最后一个子窗口可能更短。除非另有说明，所有方法共享同一个冻结的回答模型，即启用思考模式的 Qwen3-32B，从而使性能差异仅反映记忆质量。

指标。 我们的主要指标是 LLM-as-a-judge（LJ）分数 (Zheng 等, 2023)。具体而言，DeepSeek-V4-Pro 将每个预测与参考答案进行比较，并赋予二元正确性标签。我们报告每个数据集上的平均 LJ 分数，同时给出按问题类型划分的明细结果。对于效率分析（RQ2），我们测量两个量：在线端到端单查询时延，以及回答上下文记忆词元数。后者定义为方法生成并提供给回答模型的记忆块中的词元数量，并在所有问题上取平均。该计数不包括查询和固定的答案生成提示，使用 Qwen3-32B 的分词器计算。

[1] https://github.com/dorianbrown/rank_bm25
[2] https://huggingface.co/BAAI/bge-reranker-v2-m3

表 1: LongMemEval 和 LoCoMo 上的准确率。对于 LongMemEval，KU、MS、SSA、SSP、SSU 和 TR 分别表示知识更新、多会话推理、单会话助手、单会话偏好、单会话用户和时间推理。对于 LoCoMo，MH、Temp.、Open 和 Single 分别表示多跳、时间、开放域和单跳问题。LJ 表示以大语言模型为评判器的总体得分，MPM 表示记忆处理模型。MemSkill 使用在 LoCoMo 上训练的控制器，并以 Qwen3-32B 作为 MPM；MemT 则使用一个在 LoCoMo 上训练的 4B 模型作为 MPM 和答案生成模型。因此，LongMemEval 对这两种方法都构成域外评估。LoCoMo 未定义 Oracle Session。在非预言机方法中，最佳和次佳结果分别以粗体和下划线标出。

Method	**MPM**	**LongMemEval**							**LoCoMo**				
		KU	**MS**	**SSA**	**SSP**	**SSU**	**TR**	**LJ**	**MH**	**Temp.**	**Open**	**Single**	**LJ**
Gold Context													
Oracle Turn	None	0.88	0.85	0.91	0.67	0.93	0.69	0.82	0.67	0.60	0.60	0.86	0.75
Oracle Session	None	1.00	0.81	1.00	0.67	1.00	0.65	0.84	–	–	–	–	–
Basic Retrieval													
RAG Top20	None	0.81	0.63	**1.00**	0.33	**1.00**	0.54	0.71	0.49	0.51	0.75	**0.81**	0.67
RAG Top50	None	0.88	0.52	**1.00**	0.50	**1.00**	0.73	0.75	0.48	0.46	0.65	**0.81**	0.66
Write-Time Memory Construction (Training-Free)													
LightMem	Qwen3-32B	0.81	0.81	0.27	0.67	**1.00**	0.88	0.79	0.38	0.48	0.40	0.52	0.47
StructMem	Qwen3-32B	0.81	0.81	0.36	**0.83**	**1.00**	**0.92**	0.82	0.38	0.55	0.50	0.57	0.52
Mem0	Qwen3-32B	0.63	0.67	0.64	**0.83**	0.93	0.42	0.64	0.42	0.15	0.60	0.57	0.45
MemoryBank	Qwen3-32B	0.88	0.56	**1.00**	0.33	0.79	0.58	0.68	0.16	0.42	0.40	0.51	0.41
Write-Time Memory Construction (Trained)													
MemSkill	Controller + Qwen3-32B	0.81	0.41	0.55	0.50	0.64	0.58	0.57	0.36	0.43	0.50	0.44	0.43
MemT	Trained 4B	0.63	0.52	0.91	0.17	0.93	0.50	0.61	0.36	**0.58**	0.40	0.68	0.57
Query-Time Memory Construction													
NanoMemory	Qwen3-32B	0.69	0.78	**1.00**	0.33	**1.00**	0.81	0.80	0.46	0.57	0.80	**0.81**	0.68
Query-Time Memory Construction: Our Method													
LazyMem	Qwen3-32B	**1.00**	**0.93**	**1.00**	0.67	**1.00**	0.88	**0.93**	**0.58**	0.55	**0.85**	0.78	**0.69**
LazyMem	LazyMem-4B	0.94	0.67	**1.00**	0.67	**1.00**	0.88	0.85	0.55	**0.58**	0.75	0.78	0.68

4.2 主要结果（RQ1）

表 1 对 RQ1 给出了肯定回答：LazyMem 在域内与域外均取得最佳总体性能。在 LongMemEval 上，采用 Qwen3-32B 的 LazyMem 达到 0.93 的 LJ 分数，比最强的非预言机基线（StructMem，0.82）高出 0.11；LazyMem-4B 也以 0.85 的成绩优于所有基线。在 LoCoMo 上，32B 变体仍以 0.69 领先，4B 变体在未使用任何 LoCoMo 训练数据的情况下以 0.68 紧随其后，且超过了在 LoCoMo 上训练过的 MemT 与 MemSkill。在两个基准上，最强的查询时方法均优于最强的写入时方法。

提升源自何处，以及为何能够超过预言机上下文。 性能提升集中在聚合类和时间类问题上。在 KU 和 MS 上，32B 变体分别达到 1.00 和 0.93，均比最佳基线高 0.12。在 MS 和 TR 上，它甚至超过了两个金标准上下文预言机，而后两者自身的分数远未饱和（Oracle Turn：0.85/0.69；Oracle Session：0.81/0.65）。这是因为单个预言机轮次或会话无法包含散布在多个会话中的证据，而且提供完整会话实际上还会引入噪声。查询时构建会依据问题对检索到的上下文进行条件化筛选，在保留跨会话证据供下游推理使用的同时去除噪声，从而规避上述两个问题。对于单事实类型（KU、SSA、SSU），预言机上下文已足够充分，LazyMem 与预言机的成绩都在 1.00 附近饱和。在 SSP 上，两个 LazyMem 变体均达到与预言机相同的 0.67，但落后于最强的写入时方法（StructMem 和 Mem0，二者均为 0.83）。这类问题要求基于用户历史给出推荐，因此持久化用户画像可能比按需组装证据更有优势。我们在第 4.4 节进一步考察训练如何影响这种对偏好敏感的构建过程。

轻量级 4B 变体在 LongMemEval 的六类问题中有五类位列前二；唯一的例外是 MS，而 32B→4B 的总体下降（0.93→0.85）也几乎完全来自该类型（0.93→0.67）。我们在第 4.5 节结合错误归因分析这一 MS 差距。

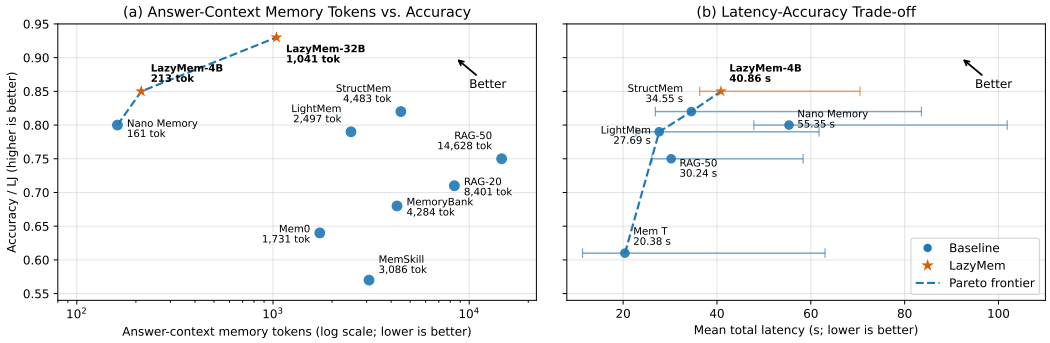


图 2: LongMemEval 上的准确率–效率权衡。左图：LJ 与平均回答上下文记忆词元数（对数刻度）的关系；由于缺少可比较的词元计数，故省略 MemT。右图：相同服务设置下，LJ 与平均端到端时延的关系；横条覆盖 P50–P95，且 LazyMem 使用 4B 记忆处理模型。虚线表示帕累托前沿（越靠左上越好）。标签分别报告词元数（左图）和平均时延（右图）。

4.3 效率分析（RQ2）

图 2 表明，LazyMem 在准确率与上下文开销之间实现了良好权衡，两种配置均位于回答上下文记忆词元数–准确率帕累托前沿上。LazyMem-4B 仅使用平均 213 个回答上下文记忆词元便取得 0.85 的 LJ；与 RAG Top50 和 StructMem 相比，该词元数分别减少了 68.7 倍和 21.0 倍，同时 LJ 分别提高 0.10 和 0.03。使用 Qwen3-32B 作为记忆处理模型后，LJ 进一步提升至 0.93，对应 1,041 个回答上下文记忆词元。这些结果表明，查询条件化构建能够在保持较强下游准确率的同时，大幅减少传递给回答模型的上下文。

时延结果显示，LazyMem-4B 在准确率与时延之间同样实现了有利权衡。尽管其在线时延高于写入时方法 LightMem 和 StructMem，但它取得了更高的准确率。与另一种查询时构建方法 NanoMemory 相比，LazyMem-4B 将 LJ 从 0.80 提升至 0.85，同时将平均端到端时延从 55.35 s 降至 40.86 s，因而位于时延–准确率帕累托前沿上。其经过训练的轻量级记忆处理模型与并行窗口处理，使额外的查询时开销保持在可控范围内。

4.4 消融研究（RQ3）

表 2: 历史窗口半径 w 的消融实验。LJ 表示答案准确率，All@50 表示用相邻轮次扩充检索到的 top-50 消息后，完整金标准证据被覆盖的问题比例。每列最佳结果以粗体标出。

Radius w	**LongMemEval**		**LoCoMo**	
	LJ	**All@50**	**LJ**	**All@50**
0	0.77	0.950	0.63	0.781
1	0.82	0.970	0.64	0.847
2	**0.85**	**0.980**	0.68	0.889
3	**0.85**	**0.980**	**0.69**	**0.892**

表 2 报告了历史窗口半径 w 的影响。增大 w 会提高证据覆盖率，但当 $w \geq 2$ 时，LongMemEval 的 LJ 饱和于 0.85，这是因为 All@50 已达到 0.980，相邻上下文能提供的额外帮助空间很小。LoCoMo 的召回率较低（0.889），因而从 w=3 中获得轻微收益（0.68→0.69），但相邻上下文仍无法找回检索区域之外的证据。

表 3 表明，逐步叠加训练相较于仅使用提示，能持续提升性能。最终的 RL 模型将 LongMemEval 上的 LJ 从 0.41 提升至 0.85，并将 LoCoMo 上的 LJ 从 0.56 提升至 0.68。SFT 贡献了大部分初始增益（LJ 分别 +0.32 / +0.07），这主要源于消除了格式错误的门控输出（Gate Fmt. 分别从 0.397 跃升至 0.963、从 0.622 跃升至 0.985）。RL 在格式几乎不变的情况下进一

表 3：LazyMem-4B 记忆处理模型（MPM）在 LongMemEval 和 LoCoMo 上逐步加入训练阶段的消融结果。Qwen3-4B 是仅使用提示词的基础 MPM，SFT 加入监督微调，RL 则进一步采用 GRPO 训练。Gate Fmt. 表示输出中包含可解析 JSON 决策数组且每条输入消息恰有一个决策的比例；它衡量结构合规性，而非决策正确性。

MPM	LongMemEval								LoCoMo					
	KU	MS	SSA	SSP	SSU	TR	Gate Fmt.	LJ	MH	Temp.	Open	Single	Gate Fmt.	LJ
Qwen3-4B	0.63	0.19	0.55	0.17	0.71	0.35	0.397	0.41	0.39	0.45	0.70	0.66	0.622	0.56
Qwen3-4B (SFT)	0.94	0.56	1.00	0.17	1.00	0.65	0.963	0.73	0.46	0.52	0.75	0.74	0.985	0.63
Qwen3-4B (RL)	0.94	0.67	1.00	0.67	1.00	0.88	0.997	0.85	0.55	0.58	0.75	0.78	0.997	0.68

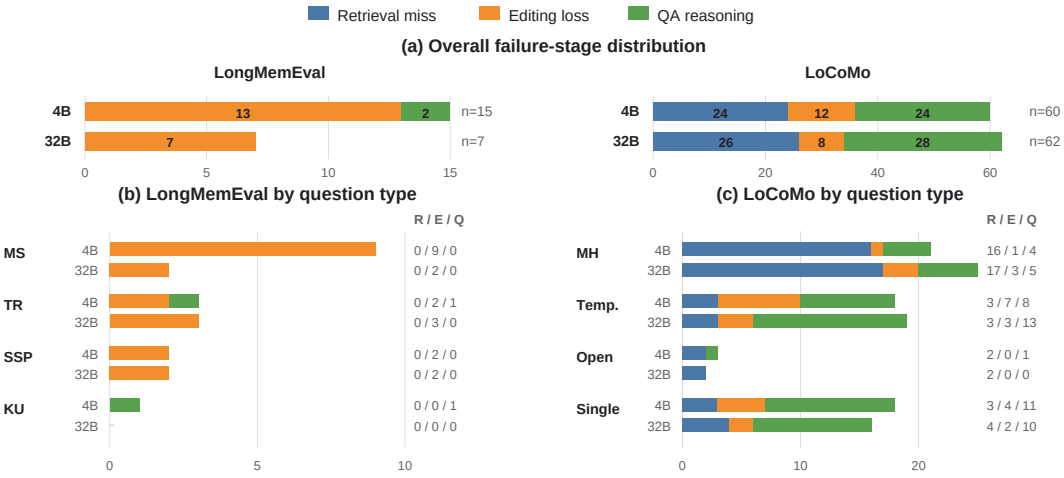


图 3：LazyMem-4B 与仅使用提示的 32B 变体中，由人工归因的总体错误及各问题类型错误。三元组表示检索遗漏 / 编辑损失 / 问答推理（R/E/Q）。SSA 和 SSU 的错误数为零。

步带来 +0.12 / +0.05 的增益，说明其增益源自记忆构建决策的改善，而非格式合规性的提高。这验证了分阶段设计：SFT 建立可靠的门控执行能力，RL 则精炼应保留的内容。

作为第 4.2 节跨方法比较的补充，SSP 呈现出独特的分阶段模式：SFT 后准确率仍为 0.17，但 RL 后升至 0.67。由于这类问题需要将用户历史转化为与推荐相关的证据，成功可能不仅取决于保留相关证据，还取决于如何压缩这些证据。因此，效用奖励可能会让压缩后的偏好更有助于当前推荐，从而改善查询时记忆构建。鉴于 SSP 子集相对较小，这一解释仍是初步的。

4.5 进一步分析（RQ4）

三名专家标注者通过多数投票，将每个已确认错误归入最先发生故障的阶段：若金标准证据从未进入候选集，则为检索遗漏；若证据在构建期间被丢弃或扭曲，则为编辑损失；若记忆已足够充分但回答模型仍失败，则为问答推理错误。仅在该归因分析中，我们排除了经人工确认的评判器噪声和金标准标注有误的样本；因此，归因错误总数不一定等于原始 LJ 所隐含的错误数。

总体分布。 在 LongMemEval 上，All@50 为 0.98；在保留的归因样本集中，不存在检索遗漏。在 15 个 4B 错误中，13 个属于编辑损失；7 个 32B 错误则全部属于编辑损失；其余两个 4B 错误出现在问答阶段。错误总数从 15 降至 7，这表明模型规模扩大的收益集中在证据编辑，而非检索或下游回答。在 LoCoMo（All@50 = 0.89）上，对于 4B，检索错误与问答错误各占 24/60；对于 32B，二者分别占 26/62 和 28/62。尽管扩大模型规模将编辑损失从 12 个降至 8 个，检索和问答仍是主要瓶颈：多跳问题主要受检索限制，而时间类与单跳问题则更常在编辑或问答阶段失败。

4B 模型在多会话问题上的差距。 32B→4B 的性能下降集中在 MS 上（0.93→0.67）。当检索固定时，编辑损失从 2 个增至 9 个。这些失败并非二元的 Keep/Drop 错误，而是压缩过程中的信息损失：4B 模型虽保留了正确消息，却遗漏了一些单独看来并不重要、但回答模型聚合跨会话证据时不可或缺的细节。32B 变体具有更强的生成能力，能够产生更全面的压缩，从而避免这一失败模式。同样，RL 在 MS 上带来的增益小于在 TR 上的增益；在 TR 中，对答案至关重要的细节更为显著，现有奖励也能为选择与压缩提供更清晰的指导。然而在 MS 上，即便压缩结果在消息层面忠实且有用，仍可能遗漏跨会话聚合所需的细节，这说明仅凭这些信号无法确保"证据完整性"。

5 结论

本工作表明，记忆构建的时机与其方式同样重要。LazyMem 将核心设计问题从存储什么重新表述为呈现什么，并将所有有损转换推迟到查询时，使每项保留决策都以下游需求为条件。这使召回–噪声权衡不再是固定的架构承诺，而转化为针对每个查询的优化问题。我们的实验表明，这种推迟既有效又实用。在 LongMemEval 上，LazyMem-4B 仅使用 213 个答案上下文记忆词元，便取得 0.85 的 LLM 裁判准确率；其得分比 RAG Top50 高 0.10，同时词元数量少 68.7×。它还可以在不进行目标领域训练的情况下迁移到 LoCoMo，并相较此前的查询时构建基线实现更优的准确率–延迟权衡。

6 伦理考量

我们的工作仅使用合成的和公开发布的对话基准；不收集或存储真实用户数据。记忆构建模型会继承源历史记录以及用于标注和奖励评判的大语言模型中存在的偏差。所有人工标注均由知晓研究目的并获得报酬的专家标注员完成。由于真实部署中的原始历史记录可能包含敏感信息，系统在存储或处理记忆之前，应获得用户同意、实施访问控制与保留期限，并支持删除操作。

参考文献

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, 和 Deshraj Yadav. Mem0：利用可扩展长期记忆构建生产就绪的 ai 智能体. *arXiv* 预印本 *arXiv:2504.19413*, 2025.

Gordon V. Cormack, Charles L A Clarke, 和 Stefan Buettcher. 倒数排序融合优于 condorcet 方法和单独的排序学习方法. 载于第 *32* 届 *ACM SIGIR* 信息检索研究与发展国际会议论文集, SIGIR '09, 第 758 – 759 页, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605584836. doi: 10.1145/1571941.1572114. URL https://doi.org/10.1145/1571941.1572114.

DeepSeek-AI, Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chengyu Hou, Chenhao Xu, Chenze Shao, Chong Ruan, Conner Sun, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Donghao Li, Dongjie Ji, Erhang Li, Fang Wei, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei, Haotian Yuan, Haowei Zhang, Haowen Luo, Haoyu Chen, Haozhe Ji, Hengqing Zhang, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J Yang, JQ Zhu, Jia Luo, Jia Song, Jia Yu, Jialiang Huang, Jialu Cai, Jian Liang, Jiangting Zhou, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jieyu Yang, Jin Chen, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jingzi Zhou, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junmin Zheng, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Leyi Xia, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Linyan Zhu, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, MS Di, MY Xu, Max Mei, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Mingxu Zhou, Minmin Han, Ning Wang, Panpan Huang, Panpan Wang, Peixin Cong, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, Ruyi Chen, SH Liu, Shanghao Lu, Shangmian Sun,

Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoqing Wu, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W Zhang, WL Xiao, Wangding Zeng, Wei An, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wenqing Hou, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyu Zhang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xueyin Chen, Xuheng Lin, Xuwei Fu, YC Yan, YQ Wang, YW Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Xu, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Shao, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Yiling Ma, Ying He, Ying Tang, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, YuKun Li, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yuduan Wang, Yuehan Yang, Yuer Xu, Yuhan Wu, Yuhao Meng, Yuheng Zou, Yukun Zha, Yunfan Xiong, Yupeng Chen, Yuping Lin, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yuting Yan, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, ZF Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zekai Zhang, Zhangli Sha, Zhe Fu, Zhe Ju, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zheren Gao, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhongyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihua Qu, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Ziyi Wan, Zizheng Pan, 和 Zongqing Yao. Deepseek-v4：迈向高效的百万词元上下文智能, 2026. URL https://arxiv.org/abs/2606.19348.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, 和 Jonathan Larson. 从局部到全局：面向查询摘要的图 rag 方法. *arXiv* 预印本 *arXiv:2404.16130*, 2024.

Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, 和 Ningyu Zhang. Lightmem：轻量高效的记忆增强生成. 载于第十四届国际学习表征会议, 2026. URL https://openreview.net/forum?id=dyJ0GWpjJB.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, 和 Wen-tau Yih. 面向开放域问答的稠密段落检索. 载于 Bonnie Webber, Trevor Cohn, Yulan He, 和 Yang Liu（编）, *2020* 年自然语言处理经验方法会议（*EMNLP*）论文集, 第 6769–6781 页, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.550. URL https://aclanthology.org/2020.emnlp-main.550/.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, 和 Douwe Kiela. 面向知识密集型自然语言处理任务的检索增强生成. 载于 H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, 和 H. Lin（编）, 神经信息处理系统进展, 第 33 卷, 第 9459–9474 页. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

Kai Li, Xuanqing Yu, Ziyi Ni, Yi Zeng, Yao Xu, Zheqing Zhang, Xin Li, Jitao Sang, Xiaogang Duan, Xuelei Wang, Chengbao Liu, 和 Jie Tan. TiMem：面向长程对话智能体的时间层次化记忆巩固. 载于 Maria Liakata, Viviane P. Moreira, Jiajun Zhang, 和 David Jurgens（编）, 计算语言学协会论文集：*ACL 2026* 论文精选, 第 21700–21720 页, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.1091. URL https://aclanthology.org/2026.findings-acl.1091/.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, 和 Percy Liang. 迷失在中间：语言模型如何使用长上下文. 计算语言学协会汇刊, 12:157–173, 2024.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, 和 Yuwei Fang. 评估大语言模型智能体的超长期对话记忆. 载于第 *62* 届计算语言学协会年会论文集（第一卷：长文）, 第 13851–13870 页, 2024.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, 和 Joseph E. Gonzalez. Memgpt：迈向作为操作系统的大语言模型, 2024. URL https://arxiv.org/abs/2310.08560.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, 和 Michael S. Bernstein. 生成式智能体：人类行为的交互式拟像. 载于第 *36* 届 *ACM* 用户界面软件与技术年会论文集, UIST '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701320. doi: 10.1145/3586183.3606763. URL https://doi.org/10.1145/3586183.3606763.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, 和 Daya Guo. Deepseekmath：推动开放语言模型数学推理的极限, 2024. URL https://arxiv.org/abs/2402.03300.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, 和 Chuan Wu. Hybridflow：灵活高效的 rlhf 框架. 载于第二十届欧洲计算机系统会议论文集, EuroSys '25, 第 1279–1297 页. ACM, March 2025. doi: 10.1145/3689031.3696075. URL http://dx.doi.org/10.1145/3689031.3696075.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, 和 Denny Zhou. 大语言模型很容易受到无关上下文干扰. 载于国际机器学习会议, 第 31210–31227 页. PMLR, 2023.

Haoran Sun 和 Shaoning Zeng. 面向大语言模型智能体高效长期推理的层次化记忆. *arXiv* 预印本 *arXiv:2507.22925*, 2025.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, 和 Dong Yu. Longmemeval：评测聊天助手的长期交互记忆. 载于第十三届国际学习表征会议, 2025. URL https://openreview.net/forum?id=pZiyCaVuti.

Yuqian Wu, Wei Chen, Zhengjun Huang, Junle Chen, Qingxiang Liu, Kai Wang, Xiaofang Zhou, 和 Yuxuan Liang. 回归基础：仅凭检索与生成让对话智能体拥有记忆. *arXiv* 预印本 *arXiv:2604.11628*, 2026.

Buqiang Xu, Yijun Chen, Jizhan Fang, Ruobin Zhong, Yunzhi Yao, Yuqi Zhu, Lun Du, 和 Shumin Deng. Structmem：面向大语言模型长程行为的结构化记忆. 载于第 *64* 届计算语言学协会年会论文集（第二卷：短文）, 第 122–146 页, 2026.

Fangyuan Xu, Weijia Shi, 和 Eunsol Choi. RECOMP：通过上下文压缩与选择性增强改进检索增强语言模型. 载于第十二届国际学习表征会议, 2024. URL https://openreview.net/forum?id=mlJLVigNHp.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, 和 Yongfeng Zhang. A-mem：面向大语言模型智能体的智能体式记忆. 载于 D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, 和 N. Chen（编）, 神经信息处理系统进展, 第 38 卷, 第 17577–17604 页. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/19909c36f51abc4856b4560aff3d36d6-Paper-Conference.pdf.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z Pan, 等. Memory-r1：通过强化学习增强大语言模型智能体管理和利用记忆的能力. 载于第 *64* 届计算语言学协会年会论文集（第一卷：长文）, 第 12805–12825 页, 2026.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui

Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, 和 Zihan Qiu. Qwen3 技术报告, 2025. URL https://arxiv.org/abs/2505.09388.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, juncai liu, LingJun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Ru Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Yonghui Wu, 和 Mingxuan Wang. Dapo：大规模开源大语言模型强化学习系统. 载于 D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, 和 N. Chen（编）, 神经信息处理系统进展, 第 38 卷, 第 113222–113244 页. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/a4277440d50f1f15d2cb4c14f7e0c0d2-Paper-Conference.pdf.

Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, 和 Bryan Catanzaro. Rankrag：统一大语言模型中的上下文排序与检索增强生成. 神经信息处理系统进展, 37:121156–121184, 2024.

Yanwei Yue, Guibin Zhang, Boci Peng, Xuanbo Fan, Jiaxin Guo, Qiankun Li, 和 Yan Zhang. Memt：为长程记忆智能体提供稠密奖励. 载于第四十三届国际机器学习会议, 2026. URL https://openreview.net/forum?id=8ppVmLtA2V.

Guibin Zhang, Haotian Ren, Chong Zhan, Junhao Wang, He Zhu, Wangchunshu Zhou, 和 Shuicheng YAN. Memevolve：智能体记忆系统的元进化. 载于第四十三届国际机器学习会议, 2026a. URL https://openreview.net/forum?id=qpkG0eKx4v.

Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, 和 Wenya Wang. Memskill：为自进化智能体学习并演化记忆技能. *arXiv* 预印本 *arXiv:2602.02474*, 2026b.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, 和 Jingren Zhou. Qwen3 embedding：通过基础模型推进文本嵌入与重排序, 2025. URL https://arxiv.org/abs/2506.05176.

Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, Yuandong Tian, 和 Jishen Zhao. Amabench：评估智能体应用的长程记忆. 载于国际机器学习会议（*ICML*）, 2026.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, 等. 使用 mt-bench 和 chatbot arena 评判“大语言模型作为评判器”. 神经信息处理系统进展, 36:46595–46623, 2023.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, 和 Zheyan Luo. LlamaFactory：统一高效微调 100 多种语言模型. 载于 Yixin Cao, Yang Feng, 和 Deyi Xiong（编）, 第 *62* 届计算语言学协会年会论文集（第三卷：系统演示）, 第 400–410 页, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-demos.38. URL https://aclanthology.org/2024.acl-demos.38/.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, 和 Yanlin Wang. Memorybank：用长期记忆增强大语言模型. 载于 *AAAI* 人工智能会议论文集, 第 38 卷, 第 19724–19731 页, 2024.

A AMA-Bench 针式诊断

为理解记忆增强系统如何丢失信息，我们报告 Zhao 等 (2026) 提出的针式诊断。表 4 复现原工作报告的结果；我们没有重新运行相应实验。该诊断在三种输入条件下评估相同的含针样例，三者仅在答案模型可见的信息上有所不同：包含针信息的原始观测、由这些观测构建的记忆，以及完整检索流程返回的记忆。从包含针信息的完整观测到包含针信息的已构建记忆的下降，衡量了检索介入之前由记忆构建引入的信息损失。四种方法的构建损失都很显著，从 HippoRAG2 的 19.6% 到 MemoryBank 的 41.3% 不等，这说明将原始观测转换为持久记忆时，相关证据可能被遗漏、压缩或改变。

表 4: AMA-Bench 上的针式诊断结果，复现自 Zhao 等 (2026)。包含针信息的完整观测向答案模型提供含目标证据的原始观测轮次；包含针信息的已构建记忆则提供由这些观测构建的、方法特定的记忆，从而隔离记忆构建过程中的信息损失。端到端系统还应用各方法的检索流程，以捕捉检索造成的进一步损失。括号内的百分比表示相对于紧邻前一列的相对下降幅度。

Method	**Full Obs. w/ Needle**	**Constructed Mem. w/ Needle**	**End-to-End System**
HippoRAG2	0.46	0.37 (↓ 19.6%)	0.21 (↓ 43.2%)
Mem1	0.46	0.29 (↓ 37.0%)	0.20 (↓ 31.0%)
A-Mem	0.46	0.29 (↓ 37.0%)	0.24 (↓ 17.2%)
MemoryBank	0.46	0.27 (↓ 41.3%)	0.26 (↓ 3.7%)

B NanoMemory 中检索深度与模型规模的影响

表 5 和表 6 分别研究 NanoMemory 中检索深度与构建模型规模对 LongMemEval 和 LoCoMo 的影响。LongMemEval 的时延测量遵循第 C.2 节的统一协议。NanoMemory 在一次构建调用中处理整个检索会话池；若不改变方法，便无法将其拆分为独立的并行调用，因此其批大小和并发数均为 1。

增大 top-k 可提高证据覆盖率，但也会引入更多需要构建模型过滤的词元。在 LongMemEval 上，32B 模型从 Top-3 时的 0.71 提升到 Top-10 时的 0.80，而 4B 模型在 Top-5 达到峰值后开始下降。LoCoMo 上也出现类似模式：32B 模型最高提升至 0.73，而 4B 模型在 Top-20 达到峰值，并在更深检索下退化。这说明规模更大的构建模型能更有效地处理更长、噪声更多的检索上下文。

过滤质量的提升以更高计算成本为代价。在 LongMemEval 上，32B 模型的平均过滤时延为 26.64–40.74 秒，而 4B 模型为 16.95–19.54 秒。在检索上下文最长的 Top-10 处，性能差距最大。因此，NanoMemory 在检索覆盖、过滤质量、上下文规模与时延之间呈现清晰权衡。

C 更多效率结果

C.1 答案上下文的记忆词元数

表 7 报告图 2 中 LongMemEval 结果所对应的精确答案上下文记忆词元数。LazyMem-4B 仅提供 213 个词元，比 RAG Top50 少 68.7×，比 StructMem 少 21.0×，同时取得更高的 LJ 准确率。仅使用提示词的 32B 变体使用 1,041 个词元，LJ 达到 0.93，体现了构建能力与上下文效率之间的权衡。为完整起见，我们也给出 LoCoMo 的统计，其在两种 LazyMem 配置之间呈现相同的定性差异。

C.2 时延协议与分解

所有时延实验均在配备两张 NVIDIA A800 80GB PCIe GPU 和 AMD EPYC 7763 CPU 的服务器上运行。我们在一张 GPU 上使用 vLLM 以 bfloat16 精度和 0.9 的 GPU 显存利用率提供大

表 5: NanoMemory 中检索深度与记忆构建模型规模对 LongMemEval 的影响。答案模型固定为 Qwen3-32B，构建模型表示用于查询条件化记忆构建的模型。*Tok.* 是传给记忆构建模型的平均检索词元数，使用 Qwen3-32B 分词器计算。时延仅报告记忆过滤与构建，不包括检索和答案生成。

			LongMemEval							**Filtering Latency (s)**			
Top-k	Constructor	Tok.	KU	MS	SSA	SSP	SSU	TR	LJ	Mean	Median	P90	P95
3	Qwen3-4B	7808	0.63	0.44	0.82	0.33	0.71	0.58	0.58	17.55	8.61	28.74	43.06
	Qwen3-32B		0.88	0.56	1.00	0.33	0.79	0.69	**0.71**	26.64	21.66	46.08	59.72
5	Qwen3-4B	12167	0.75	0.48	1.00	0.50	0.86	0.58	0.66	16.95	9.81	26.07	39.56
	Qwen3-32B		0.75	0.52	1.00	0.50	0.86	0.73	**0.71**	38.14	24.80	58.60	67.61
10	Qwen3-4B	23841	0.69	0.52	1.00	0.33	0.86	0.54	0.64	19.54	12.50	31.90	43.63
	Qwen3-32B		0.69	0.78	1.00	0.33	1.00	0.81	**0.80**	40.74	32.54	67.71	87.49

表 6: NanoMemory 中检索深度与记忆构建模型规模对 LoCoMo 的影响。答案模型固定为 Qwen3-32B。由于 LoCoMo 的会话更短，因此采用更大的 top-k 值；Top-30 的词元预算与 LongMemEval Top-10 相当。

Top-k	Constructor	Tok.	Multi	Temp.	Open	Single	LJ
3	Qwen3-4B	2646	0.23	0.20	0.45	0.64	0.45
	Qwen3-32B		0.29	0.38	0.60	0.68	**0.53**
5	Qwen3-4B	4281	0.28	0.29	0.65	0.70	0.52
	Qwen3-32B		0.33	0.51	0.70	0.73	**0.59**
10	Qwen3-4B	8260	0.39	0.42	0.70	0.76	0.60
	Qwen3-32B		0.46	0.57	0.80	0.81	**0.68**
15	Qwen3-4B	12084	0.42	0.37	0.65	0.78	0.61
	Qwen3-32B		0.46	0.60	0.80	0.83	**0.70**
20	Qwen3-4B	15888	0.42	0.43	0.50	0.84	0.64
	Qwen3-32B		0.57	0.54	0.85	0.82	**0.71**
25	Qwen3-4B	19579	0.36	0.35	0.60	0.78	0.59
	Qwen3-32B		0.52	0.58	0.85	0.87	**0.73**
30	Qwen3-4B	21512	0.35	0.29	0.60	0.76	0.56
	Qwen3-32B		0.55	0.54	0.80	0.87	**0.73**

语言模型推理服务，并在另一张 GPU 上计算问题嵌入。每种方法使用其各自配置的模型规模。

我们严格串行处理问题，不在问题之间进行批处理或并行。对于当前问题，LazyMem 并发发出相互独立的历史窗口请求（一次最多 64 个）。相比之下，NanoMemory 在一次调用中处

表 7：答案上下文的记忆词元数，定义为每个问题中，由方法生成并提供给答案模型的记忆块的平均词元数。统计不包括查询和固定的答案生成提示词，并使用 Qwen3-32B 分词器计算。LongMemEval 列对应图 2 中答案上下文记忆词元数与准确率的结果；为完整起见，同时给出 LoCoMo 列。N/A 表示该方法未提供在我们的统计协议下可直接比较的答案上下文记忆词元数。

Method	**Constructor**	**LongMemEval**	**LoCoMo**
	Gold Context		
Oracle Turn	None	515	171
Oracle Session	None	5,778	–
	Basic Retrieval		
RAG Top20	None	8,401	1,658
RAG Top50	None	14,628	3,834
	Write-Time Construction (Training-Free)		
LightMem	Qwen3-32B	2,497	2,445
StructMem	Qwen3-32B	4,483	5,010
Mem0	Qwen3-32B	1,731	8,881
MemoryBank	Qwen3-32B	4,284	1,599
	Write-Time Construction (Trained)		
MemSkill	Trained Controller + Qwen3-32B	3,086	2,358
MemT	Trained 4B	N/A	N/A
	Query-Time Construction		
NanoMemory	Qwen3-32B	161	133
	Query-Time Construction: Our Method		
LazyMem	Qwen3-32B	1,041	1,945
LazyMem	LazyMem-4B	213	697

表 8：LongMemEval 上的端到端时延分解。总时延包括检索、方法特定的记忆处理和答案生成，不包括写入时方法的离线记忆构建成本。“处理”表示检索后执行的记忆过滤或构建。Med.、P90 和 P95 分别表示中位数、第 90 百分位数和第 95 百分位数。破折号表示相应阶段不适用。

Method	Total Latency (s)				Retrieval Latency (s)				Processing Latency (s)				Answer Latency (s)			
	Mean	Med.	P90	P95	Mean	Med.	P90	P95	Mean	Med.	P90	P95	Mean	Med.	P90	P95
RAG Top50	30.24	26.13	50.99	58.36	1.58	1.53	1.89	1.97	–	–	–	–	28.66	24.35	49.41	56.45
MemT	20.38	11.35	44.48	63.03	14.58	7.53	31.68	35.54	–	–	–	–	5.80	3.46	9.77	17.37
LightMem	27.69	22.73	49.69	61.76	0.04	0.02	0.07	0.08	–	–	–	–	27.65	22.67	49.67	61.73
StructMem	34.55	26.87	69.14	83.58	0.04	0.03	0.04	0.05	–	–	–	–	34.51	26.84	69.08	83.55
NanoMemory	55.35	47.85	89.39	101.86	0.05	0.05	0.06	0.06	40.74	32.54	67.71	87.49	14.56	12.67	22.01	26.84
LazyMem-4B	40.86	36.31	61.86	70.49	1.58	1.53	1.89	1.97	23.09	20.83	36.26	41.83	16.18	12.78	28.84	36.70

理整个检索池；若不改变其流程，便无法提供可比的查询内并行。因此，时延结果包含了 LazyMem 的这一结构性优势。我们舍弃三次预热运行，并报告随后三次运行的统计结果。

表 8 将端到端时延分解为检索、记忆处理和答案生成。查询时方法增加了一个记忆处理阶段。LazyMem 行使用训练后的 4B 记忆处理模型，而 NanoMemory 使用 Qwen3-32B，与正文效率分析中比较的配置一致。

D 实验设置细节

D.1 数据集与划分构建

LongMemEval (Wu 等, 2025) 包含 500 个问题，每个问题都基于一段很长的多会话用户–助手交互历史。该数据集定义了六种问题类型：单会话用户、单会话助手、单会话偏好、多会话、时间推理和知识更新。弃答问题通过问题 ID 中的后缀 `_abs` 标识，并保留其原始问题类型。

由于我们的方法需要训练数据，我们按类型对这 500 个问题进行分层，并划分为 360/40/100 个训练、验证和测试样例（72%/8%/20%）。在每种类型内，样例先按问题 ID 排序，再以固定随机种子 42 打乱；同时采用最大余数法保持原始类型分布。

表 9 报告所得各类型的数量。测试划分不参与 SFT 和 RL 训练、超参数调优与检查点选择。所有模型选择决策均基于验证划分；测试划分仅用于最终评估。

表 9: LongMemEval 各划分的问题类型统计。

Question type	Total	Train	Validation	Test
Single-session user	70	50	6	14
Single-session assistant	56	40	5	11
Single-session preference	30	22	2	6
Multi-session	133	96	10	27
Temporal reasoning	133	96	11	26
Knowledge update	78	56	6	16
Total	500	360	40	100

D.2 训练数据

这里，金标表示由基准标注的必需证据；非金标表示未经标注的消息，这些消息不一定不相关，仍可能提供辅助上下文。

SFT 数据构建。 我们按问题类型从包含 360 个问题的训练划分中抽取 100 个问题，得到 3,075 个查询–窗口对。我们在高思考模式下使用代码清单 1 中的提示调用 DeepSeek-V4-Flash (DeepSeek-AI 等, 2026)。对于窗口中的每条消息，教师模型预测保留/丢弃决策，并在保留时生成压缩改写；其推理过程和结构化决策共同构成 SFT 目标。

由于金标证据在窗口中较为稀疏，原始标注中保留和丢弃的比例分别为 3.92% 和 96.08%。我们丢弃了 36 个遗漏金标证据的窗口，随后在窗口级进行平衡：将包含保留决策的 388 个窗口与采样得到的 388 个全丢弃窗口配对，最终得到 776 个实例；其平均输入长度和目标长度分别为 2,585 和 873 个词元。自动化验证未在原始标注中发现格式错误。

RL 数据平衡。 原始 RL 训练池包含 11,072 个查询–窗口提示，其中 8.98% 包含金标证据，91.02% 不包含。我们分别从两组中采样 5,000 个提示，构建包含 10,000 个提示的平衡训练集：对包含金标证据的提示进行有放回采样，对其余提示进行无放回采样。

D.3 训练超参数

表 10 列出了两个训练阶段中显式设置的配置。SFT 使用 LLaMA-Factory (Zheng 等, 2024) 0.9.3 进行全参数微调，并配合 DeepSpeed ZeRO-2 运行。RL 阶段使用由 VERL (Sheng 等, 2025) 0.7.1 结合 Ray 与 FSDP 实现的 GRPO (Shao 等, 2024)。两个阶段均在单个节点上使用两张 NVIDIA A800 80GB PCIe GPU 训练 Qwen3-4B (Yang 等, 2025)。SFT 耗时约 0.36 小时。RL 数据加载器会丢弃每个训练轮次中最后一个不完整批次，因此，包含 10,000 个提示的训练集、32 的全局批大小和三个训练轮次共同产生 $\lfloor 10{,}000/32 \rfloor \times 3 = 936$ 个优化步骤。观测到的实际时间为每次不间断 RL 运行约 7.5–8.5 天，其中包括周期性验证和检查点保存。

对于 SFT，有效全局批大小由单设备批大小 1、梯度累积步数 8 与 2 张 GPU 相乘得到。对于 RL，批大小表示每个训练批次中的提示数量；每个提示采样 8 次。RL actor 使用大小为 16 的 PPO 小批次和大小为 4 的单 GPU 微批次。采样生成使用 top-$p = 0.95$ 的核采样。

表 10: SFT 与 RL 阶段显式设置的训练超参数。

	SFT	RL (GRPO)
Base model	Qwen3-4B	SFT checkpoint
Framework	LLaMA-Factory 0.9.3	VERL 0.7.1
Learning rate	5×10^{-6}	1×10^{-6}
LR schedule	cosine	constant
Warmup ratio	0.03	0.0
Epochs	2	3
Global batch size	16	32
Dataloader `drop_last`	—	Yes
Max input length	8192	8192
Max generation length	—	3072
Rollouts per prompt	—	8
Sampling temperature (rollout)	—	0.9
KL coefficient	—	0.001
KL loss type	—	`low_var_kl`
Symmetric PPO clip ratio ϵ	—	0.2
Precision	bf16	bf16
Optimizer	AdamW	AdamW
Weight decay	—	0.01
Adam β values	—	$(0.9, 0.999)$
Gradient clipping	—	1.0
Distributed	DeepSpeed ZeRO-2	Ray + FSDP
Rollout backend	—	vLLM 0.14.0 (TP=1)
Other GRPO parameters	—	VERL 0.7.1 defaults
Hardware	2× A800 80GB PCIe	

训练节点提供八张 A800 GPU，但两个阶段均设置 `CUDA_VISIBLE_DEVICES=0,1`，每个节点使用两张 GPU（每张 81,920 MiB）。RL actor 使用参数精度为 bf16 的 FSDP，且不卸载参数或优化器。参考模型使用 FSDP 参数卸载，并禁用 Torch 编译。Ulysses 序列并行度和 vLLM 张量并行度均设为 1。RL 阶段不使用 DeepSpeed 和 ZeRO。

训练脚本并未覆盖设置单一的全局随机种子。在解析后的 VERL 配置中，FSDP 种子与 actor 数据加载器种子均为 42，而 `data.seed` 保持未设置状态。我们在训练前以及每 10 个优化步骤进行一次验证，每 30 个步骤保存一个检查点，最多保留五个 actor 检查点，并保存模型、优化器和辅助训练状态。其余所有 RL/GRPO 参数均使用 VERL 0.7.1 的默认值。

D.4 实现与推理细节

构建后的记忆与查询拼接后传递给回答模型；回答提示见代码清单 3。

基线实现。 我们遵循每种基线的原始或官方流水线进行实现，并在适当情况下统一骨干模型与检索预算。

- **Oracle Turn and Oracle Session**. 分别向回答模型提供金标证据所在的对话轮次，或包含该证据的完整会话。
- **RAG Top20 and RAG Top50**. 对于每个查询，我们使用与 LazyMem 相同的混合检索流水线（第 3.1 节）检索前 20 条或前 50 条消息，随后按时间顺序拼接，再生成答案。
- **LightMem** (Fang 等, 2026). 我们采用原始的记忆条目构建与检索流水线，并使用混合的用户–助手消息，检索前 50 个记忆条目。
- **StructMem** (Xu 等, 2026). 我们采用同时包含结构化摘要和记忆条目的原始流水线，使用混合的用户–助手消息，并检索前 50 个记忆条目和前 5 个结构化摘要。
- **Mem0** (Chhikara 等, 2025). 我们使用开源 Mem0 流水线，以 Qwen3-Embedding-8B 进行稠密检索，并以 Qdrant 作为向量存储；每个查询检索前 50 条记忆，其余设置遵循官方 Mem0 基准配置。

- **MemoryBank** (Zhong 等, 2024). 我们保留 MemoryBank 的原始设计，包括用户特定的长期记忆库、带日期的对话记忆、事件级与人格级摘要，以及使用 all-MiniLM-L6-v2 嵌入和 FAISS 索引的 Top-8 检索。这样保留了其原生记忆粒度，而非采用其他基线所使用的统一检索粒度。
- **MemSkill** (Zhang 等, 2026b). 我们使用原作者发布、在 LoCoMo 上训练的控制器检查点，不进行重新训练，直接在我们的测试划分上对其进行零样本评估。Qwen3-32B 作为执行器，并检索前 50 条记忆用于生成答案。
- **MemT** (Yue 等, 2026). 我们遵循官方推理设置，使用发布的 Mem-T-4B 检查点、默认检索 top-k，以及原始的工具调用与最大工具步数设置。对于 LongMemEval，我们使用官方流水线构建记忆库；对于 LoCoMo，我们使用已发布的预构建记忆库。
- **NanoMemory** (Wu 等, 2026). 我们使用其带有会话级检索的查询时记忆构建流水线，并将检索深度从原始的 Top-3 提高到 Top-10，以进行更强的比较。

D.5 LLM 评审器

对于 **LongMemEval** (Wu 等, 2025) 和 **LoCoMo** (Maharana 等, 2024)，我们均采用随原论文发布的官方评估脚本和评审提示，仅更换评审模型。具体而言，评审器为 DeepSeek-V4-Pro（通过兼容 OpenAI 的 API `https://api.deepseek.com/v1` 访问，访问时间为 2026 年七月，温度为 0）；该评审器接收问题、参考答案与模型预测，并输出二元正确性标签。我们不修改这些脚本中的提示、决策规则或答案匹配逻辑，只替换底层模型。

在每个基准中，该评审器统一应用于所有系统，且不会获知预测由哪个系统生成。使用相同的评审器避免了特定于方法的评估流程，但仍可能出现依赖于模型的评审错误；附录 F.2 对其进行了量化。

E 奖励细节

本附录对第 3.3 节的奖励设计作进一步补充，包括动作奖励分析（第 E.1 节）、质量奖励评分标准与评审器配置（第 E.2 节），以及奖励混合与课程设置（第 E.3 节）。

E.1 动作奖励分析

我们在所有实验中均使用 $\eta = 0.915$。这一固定的非对称权重更加重视保留金标证据，同时仍对移除非金标上下文给予正向激励。我们将其视为一种召回优先的偏好，而非对下游错误成本的校准估计。

观测到的窗口组成。 原始 RL 池在窗口之间高度不平衡：91.02% 的窗口不包含金标消息。我们通过对包含金标消息的窗口与仅含非金标消息的窗口进行等量采样，解决这种提示级不平衡。混合窗口内部同样存在不平衡，其中非金标/金标消息数量比的中位数与第 95 百分位数分别为 5 和 7。按类别聚合消除了窗口内部的这种数量占比效应。所有观测到的混合窗口最多包含八条金标消息；因此，在我们的设置下，纠正一条被丢弃的金标消息对 R_{act} 的影响，大于整个非金标类别所能贡献的最大变化。这种动作级区分在原始 RL 池、平衡 RL 集与验证集的每个混合窗口上均成立。

奖励排序诊断。 接下来，我们使用其他动作奖励对相同的采样行为重新评分。SFT 参考策略针对 218 个验证提示中的每一个生成八次采样，共得到 1,744 个输出，其中 1,675 个（96.04%）通过了训练时的格式/模式门控。该门控仅检查结构有效性，不判断动作正确性。由于两个类别准确率同向变化的样本对无法揭示它们的相对权重，我们评估通过格式检查的冲突样本对：其中一次采样保留了更多金标消息，但拒绝了更少的非金标上下文。

对于每次采样，令 A_{g} 表示被赋予 Keep 动作的金标消息比例，A_{n} 表示被赋予 Drop 动作的非金标消息比例。令 N_{g} 和 N_{n} 分别表示窗口中相应类别的消息数量。我们将两个诊断基线与训练所使用的动作奖励（式 7）进行比较：

$$R_{\text{Micro}} = \frac{N_{\text{g}} A_{\text{g}} + N_{\text{n}} A_{\text{n}}}{N_{\text{g}} + N_{\text{n}}}, \tag{10}$$

$$R_{\text{Symmetric}} = 0.5 A_{\text{g}} + 0.5 A_{\text{n}}, \tag{11}$$

$$R_{\text{Ours}} = R_{\text{act}} = \eta A_{\text{g}} + (1-\eta) A_{\text{n}}, \qquad \eta = 0.915. \tag{12}$$

表 11: 固定且格式有效的 SFT 采样上的奖励排序。逆序率与平局率在金标/非金标冲突样本对上计算（越低越好）。金标差距是保留金标效果最佳的采样与丢弃金标效果最佳的采样之间的平均奖励差（越高越好）。

Reward	Inversion	Tie	Gold gap
Micro	54.2%	45.8%	0.173
Symmetric	0.0%	16.9%	0.493
Ours	0.0%	0.0%	0.901

表 12: 验证集上的受控记忆扰动。Δ 表示相对于完整金标条件的配对准确率变化，C→I 表示从最初正确变为错误的答案比例，Tok. 表示使用 Qwen3-32B 分词器计算的平均输入长度。

Memory	Acc. (%)	Δ (pp)	C→I (%)	Tok.
Full gold	77.5	—	—	728
Drop one gold	18.0	-59.5	80.6	603
Drop all gold	10.0	-67.5	93.5	122
+1 non-gold	85.8	+8.3	3.2	956
+5 non-gold	79.2	+1.7	6.5	2,021
+all non-gold	42.5	-35.0	48.4	30,385

微观奖励即普通的消息级动作准确率，因此每个类别的贡献与其消息数量成正比。对称奖励为保留金标消息和拒绝非金标消息赋予相同权重，而我们的非对称奖励优先保留金标消息。按照构造，每个冲突样本对都包含一次 A_{g} 更高但 A_{n} 更低的采样。令 o_{g} 表示该次采样，o_{n} 表示另一次采样。当 $R(o_{\mathrm{g}}) < R(o_{\mathrm{n}})$ 时发生逆序：奖励更偏好较好的非金标拒绝，而非较好的金标保留。当 $R(o_{\mathrm{g}}) = R(o_{\mathrm{n}})$ 时发生平局，此时奖励无法在二者之间表达偏好。表 11 报告了出现每种结果的冲突样本对比例。

微观平均会使观测到的每个冲突样本对发生逆序或平局。对称类别平衡消除了逆序，但仍有 16.9% 的样本对出现平局；相比之下，我们的非对称奖励同时消除了这两种情况，并显著扩大了金标差距。该诊断支持召回优先的机制，但不能唯一确定所报告的具体数值。

下游扰动。 最后，我们检验这种偏好是否与下游回答敏感性一致。在 40 个验证问题上，我们将完整金标记忆与移除金标证据或添加非金标消息的配对变体进行比较，并采用与主要评估相同的 Qwen3-32B 回答模型和正确性评审器。我们首先在每个问题内对多种删除和随机添加情形取平均，随后通过问题级配对自助法计算置信区间。表 12 总结了由此产生的回答准确率与输入长度变化。

移除一条金标消息会使准确率下降 59.5 个百分点（配对 95% 置信区间：$[-74.2, -42.8]$），并使最初回答正确的样例中有 80.6% 变为错误。额外保留一条或五条非金标消息不会产生统计上可检出的性能下降；而保留所有局部非金标上下文会使准确率下降 35.0 个百分点（95% 置信区间：$[-52.5, -20.0]$），同时将平均输入长度增加至 30,385 个词元。单金标与多金标问题上均呈现相同的定性模式。这些干预结果支持对金标保留持强烈但非排他的偏好：遗漏证据具有高度破坏性，有限的过度保留相对可以容忍，而极端的过度保留仍然有害。

综合来看，这些诊断验证了非对称奖励的偏好方向与工作区间，而非声称 $\eta = 0.915$ 是经过校准的最优值。若要确定端到端性能对该权重精确取值的敏感性，需要在其他设置下重新训练策略。

E.2 质量奖励：评分标准与评审器配置

评审模型。 我们使用 Qwen3-32B (Yang 等, 2025) 计算质量奖励 R_{qual}（式 9），温度设为 0，最大输出长度为 1,024 个词元。思考模式已启用，预算为 512 个词元；每个请求的超时时间为 180 s，最多重试三次。在训练期间，评审器接收源消息、查询、金标答案与引用的推理链。金标答案和推理链仅在计算训练奖励时用于评估效用，在推理时不可用。完整的评审提示见代码清单 2。

忠实度得分 f_x 评判压缩内容是否以源消息为依据，而效用得分 u_x 评判该内容对回答查询 q 的帮助程度。表 13 给出了两个维度的完整三级评分标准。

表 13: 质量奖励中忠实度与效用分量的评分标准。

Dimension	Score	Criterion
Faithfulness	0	Contains unsupported, fabricated, contradictory, or over-inferred information not grounded in the source message.
	1	Mostly faithful but contains minor ambiguity, imprecise wording, or mild over-generalization not explicitly supported by the source.
	2	Fully faithful; all content is explicitly supported by the source with no contradiction, hallucination, or over-inference.
Utility	0	Irrelevant or misleading; not helpful for the reference answer or reasoning chain.
	1	Somewhat helpful; provides indirect support, useful background, disambiguating context, or partially relevant evidence.
	2	Strongly helpful; preserves key evidence, constraints, or context needed to reach, verify, or explain the answer.

人工验证。 我们从训练期间生成的采样中随机抽取 100 个消息级压缩实例。三位专家标注员使用与奖励评审器相同的评分标准，独立评估每个实例的忠实度与效用，且无法看到评审器的预测。最终人工标签由多数投票决定；若三方票数各不相同，则由第四位专家裁决。表 14 报告了奖励评审器与所得人工标签之间的一致率。

表 14: 在随机抽取的 100 个训练采样上，奖励评审器与专家聚合人工标签之间的精确匹配一致率。

Dimension	Agreement
Faithfulness	91% (91/100)
Utility	85% (85/100)

E.3 奖励混合与课程超参数

表 15 列出了训练所使用的奖励混合与课程设置。动作奖励权重编码的是对金标保留的相对偏好，而非经过校准的下游成本。

表 15: 奖励混合与课程超参数。

Component	Parameter	Value
Action reward	Gold-retention weight η	0.915
Reward mixture	Phase 1 quality weight λ_{qual}	0.0
	Phase 2 quality weight λ_{qual}	0.5
Curriculum	Format-validity threshold τ	0.99

F 错误归因与评估可靠性

本附录以具有代表性的失败示例和评估可靠性分析，补充第 4.5 节的内容。

F.1 代表性失败示例

表 16 给出了各类失败的代表性示例。前缀为 R 的案例表示检索遗漏，C 表示编辑损失，Q 表示 QA 推理错误。

这些示例体现了不同的信息瓶颈。检索遗漏会在构建前移除所需事实；编辑损失或是丢弃某个操作数，或是破坏时间锚点；而 QA 错误则发生在构建后记忆已包含充分证据的情况下。

表 16: 不同流水线阶段和基准上的代表性失败示例。

ID	Bench.	Type	Stage	Phenomenon
				Retrieval Miss
R1	LoCoMo	Single-hop	Retrieval	Gold evidence (phone navigation malfunction) absent from all retrieved candidate windows; model substitutes a semantically similar self-checkout malfunction.
R2	LoCoMo	Multi-hop	Retrieval	Two gold items needed (mansion + Ferrari); only Ferrari recalled. Model reports what it sees but cannot complete the set.
				Editing Loss
C1	LoCoMo	Temporal	Editing	"last week" rewritten to message date (June 6); gold answer is the week before. Temporal anchor destroyed by compression.
C2	LoCoMo	Temporal	Editing	"Yesterday" rewritten to message date (Oct 4) instead of Oct 3. One-day offset from conflating report time with event time.
C3	LongMemEval	Multi-session	Editing	Pre-approval amount ($350k) retained but sale price ($325k) dropped; QA cannot compute the $25k difference.
C4	LongMemEval	Multi-session	Editing	HelloFresh 40% discount retained but UberEats 20% dropped; comparison unanswerable with one operand.
				QA Reasoning Error
Q1	LoCoMo	Temporal	QA	Memory preserves "a few days ago" intact, but QA outputs the message date instead of computing the offset.
Q2	LongMemEval	Multi-session	QA	Both operands present (Alex 21, user 32); model outputs "cannot determine" instead of computing $32 - 21 = 11$.

F.2 人工评估与裁判噪声分析

为评估自动评估的可靠性，三名专家标注员分别独立审核两个基准上的每个 LLM 裁判标签，最终决定由多数投票确定。对于每个标签，标注员检查问题、参考答案、模型预测和自动判定。在确定正确性或处理疑似参考答案错误时，他们还会查阅完整交互历史以及基准提供的黄金证据。实体、数值和跨方法一致性信号用于对裁判判为正例的样本进行额外核查，而非取代人工审核。在 LoCoMo 上，我们还识别出 19 个存在黄金答案标注错误的问题；这些问题仅从阶段级错误归因分析中排除，仍保留在下文报告的 314 个问题的原始 LJ 和裁判校正 LJ 计算中。

假阴性（FN）指正确预测被判为错误；假阳性（FP）指错误预测被判为正确。表 17 报告了主评估中全部 13 种方法的结果（LoCoMo 上为 12 种，因为其中未定义 Oracle Session）。我们根据底层二元标签计算 Corrected LJ $=$ Raw LJ $+$ (FN $-$ FP)$/N$，随后分别将原始分数和校正分数四舍五入至两位小数。两列均保留 19 个存在黄金答案错误的 LoCoMo 问题，因此分母为 314；它们只从后续的阶段级错误归因中排除。

LongMemEval。 在 13 种方法 $\times$ 100 个问题 = 1,300 个标签中，我们识别出 18 个 FN 和 16 个 FP（总体噪声率为 2.62%）。假阴性与假阳性大致均衡，因此各方法的校正幅度在 -2 至 $+4$ 个百分点之间。LazyMem 的两个变体均不存在裁判噪声，其报告分数是准确的。校正后，这两个变体仍是排名最高的两个非预言机方法，因此核心比较保持不变。

LoCoMo。 在 12 种方法 $\times$ 314 个问题 = 3,768 个标签中，裁判噪声更高且具有明显方向性：294 个被翻转的标签中有 293 个是假阴性，只有 1 个是假阳性（总体噪声率为 7.80%）。因此，原始裁判系统性地低估了所有方法。该偏差在各方法间表现一致（各方法的 FN 率在 4.14% 至 11.46% 之间，净校正幅度为 $+4.14$ 至 $+11.15$ 个百分点）；尽管部分方法的两两排名在校正后有所变化，LazyMem 的两个变体仍并列取得校正后最佳非预言机 LJ。此外，我们还识别出 19 个黄金答案本身存在标注错误的问题（主体错误、日期错误或缺乏依据的时间断言）。这些问题仍计入主 LJ 分数和裁判校正 LJ 分数，但在将已确认错误归因于检索、编辑或 QA 阶段时会统一排除。

表 17: 主评估中所有方法的裁判噪声审计。FN 和 FP 分别表示以人工多数决定为准的假阴性和假阳性数量；Corr. LJ 应用了相应的标签校正。

	LongMemEval (100 questions)				**LoCoMo** (314 questions)			
Method	Raw LJ	FN	FP	Corr. LJ	Raw LJ	FN	FP	Corr. LJ
Gold Context								
Oracle Turn	0.82	3	0	0.85	0.75	24	0	0.82
Oracle Session	0.84	0	1	0.83	–	–	–	–
Basic Retrieval								
RAG Top20	0.71	2	2	0.71	0.67	20	0	0.74
RAG Top50	0.75	2	2	0.75	0.66	25	0	0.74
Write-Time Construction								
LightMem	0.79	2	1	0.80	0.47	36	1	0.58
StructMem	0.82	2	1	0.83	0.52	29	0	0.61
Mem0	0.64	1	3	0.62	0.45	35	0	0.56
MemoryBank	0.68	0	1	0.67	0.41	25	0	0.49
MemSkill	0.57	1	2	0.56	0.43	23	0	0.50
MemT	0.61	5	1	0.65	0.57	23	0	0.64
Query-Time Construction								
NanoMemory	0.80	0	2	0.78	0.68	13	0	0.72
LazyMem (32B)	0.93	0	0	0.93	0.69	18	0	0.75
LazyMem-4B	0.85	0	0	0.85	0.68	22	0	0.75

裁判–人工一致性。 以人工多数投票决定为参照，自动裁判在 LongMemEval 的 1,300 个标签中有 1,266 个与其一致（97.38%），在 LoCoMo 的 3,768 个标签中有 3,474 个与其一致（92.20%），总体上在 5,068 个标签中有 4,740 个一致（93.53%）。这些数值衡量的是自动裁判与人工多数决定之间的一致性。

裁判噪声的来源。 两个基准共有的主要噪声模式包括：

- 语义等价的答案因表层形式不同而被拒绝（例如，“3” 与 “3 场婚礼”，“45 分钟” 与 “单程 45 分钟”）。
- 关键词重叠掩盖事实错误，导致假阳性（例如，提及正确实体的同时给出错误的时间量）。
- 相对时间表达（如 “上周”、“明天”）未被识别为对应黄金日期。
- 谨慎的限定语（如 “没有给出确切日期”）导致裁判忽略此前给出的正确答案。

对报告结果的影响。 主表保留预先指定的自动评估指标，而经人工校正的结果作为敏感性分析。审计确认：（1）校正后，LazyMem 的两个变体在两个基准上仍是排名最高的两个非预言机方法；（2）裁判噪声并未夸大 LazyMem 报告的性能（LongMemEval 上不存在标签错误，LoCoMo 上只有假阴性）；（3）第 4.5 节的错误归因分析在对失败进行分类之前，排除了经人工确认的裁判噪声和 19 个黄金答案标注错误。

G 方法特定的提示与输出模式

本节给出我们的方法针对窗口标注、训练时质量评分和答案生成所引入提示的完整文本。最终答案正确性评估则使用第 D.5 节所述未经修改的官方基准提示。诸如 `{query}` 的花括号表达式是运行时占位符。源代码清单中的标题（例如 “SYSTEM PROMPT”）仅用于展示时分隔消息，其本身不会发送给模型。

G.1 窗口标注提示

代码清单 1 用于通过查询条件化的 Keep/Drop 决策标注局部时间顺序窗口。输出必须是一个 JSON 数组，其中按时间顺序为每条非空角色消息提供一个决策。

```
SYSTEM PROMPT

你是一个局部记忆编辑模型。

你将收到一个用户查询和一个按时间顺序排列的局部记忆窗口。
你的任务是仅针对该局部窗口，按照消息的原始顺序输出一系列局部编辑决策。

必须且只能返回一个有效的 JSON 数组。
数组中的每一项必须对应 previous_bridge、core_messages 和 next_bridge 中的一条非空角色消息。
数组中的每一项必须且只能包含三个字段：op、compressed_content 和 reason。

唯一允许的决策操作是 KEEP 和 DROP。
对于 KEEP，保留与查询相关且有助于回答查询的对话内容。
根据每条消息的具体情况，KEEP 可以采用原文保留、关键片段抽取或简洁压缩的形式。
对于 DROP，compressed_content 必须为空字符串。

只能使用所提供窗口中的内容。
不要输出 Markdown、解释或任何额外文本。
你的输出必须能被 json.loads() 直接解析。


USER PROMPT TEMPLATE

# 任务
给定以下查询条件化记忆窗口，生成一系列局部编辑决策。

# 输出要求
仅返回具有以下结构的 JSON 数组：
[
  {
    "op": "KEEP | DROP",
    "compressed_content": "若为 DROP，则为空字符串；否则为采用一种 KEEP 方式保留的内容：原文保留、关键
  片段抽取或简洁压缩",
    "reason": "简短的局部理由"
  }
]

# 局部编辑规则
1. 对 previous_bridge、core_messages 和 next_bridge 中的每条非空角色消息，输出数组都必须包含一个对应决
  策。
2. 决策必须遵循原始时间顺序：先 previous_bridge，再 core_messages，最后 next_bridge。
3. 每个决策必须且只能包含三个字段：op、compressed_content 和 reason。
4. op 必须为 KEEP 或 DROP。
5. KEEP 表示保留与查询相关且有助于主模型回答问题的对话内容。
6. DROP 表示该消息在局部范围内无关，不应进入最终记忆。
7. 对于 KEEP，请为每条消息选择最合适的保留方式：
   - 原文保留：当原始消息简短且直接有用时，保留原始消息文本；
   - 关键片段抽取：当消息中只有一部分相关时，仅保留与查询相关的片段；
   - 简洁压缩：当消息有用但过长或仅部分相关时，将其忠实地改写为更简短的形式。
8. 无论采用哪种 KEEP 方式，都必须忠实于源文本含义，不得引入无依据的事实。
9. 即使整个窗口都无关，仍须为每条非空角色消息输出一个决策，并将每个 op 均设为 DROP。
10. 当 op 为 DROP 时，compressed_content 必须为空字符串。
11. 如果一条消息可能包含答案证据、用户事实、时间更新，或与查询实体 / 属性相关的内容，即使相关性尚不确定，也应优
  先选择 KEEP。
12. 仅返回 JSON 数组。不要输出 Markdown、注释或额外文本。
13. 输出必须能被 json.loads() 直接解析。

# 查询
查询时间：{query_time}
查询：{query}

# 历史窗口
{window}
```

Listing 1: 用于局部窗口标注的系统消息与用户消息模板。

G.2 记忆质量奖励评审提示

代码清单 2 供 R_{qual} 使用，以分别从忠实度和效用两个维度对保留的构建记忆进行评分。

```
SYSTEM PROMPT

你是一名严格的记忆构建奖励评判器。请评估保留下来的 constructed_memory 是否
忠实且有用。必须严格区分这两个维度：faithfulness 仅评判 constructed_memory
与 source_messages 之间的关系；utility 评判 constructed_memory 对 gold_answer
和 referenced_reasoning_chain 的帮助程度。必须且只能返回一个有效的 JSON 对象。
不要输出 Markdown、解释或额外文本。


USER PROMPT TEMPLATE

{
  "query_time": "{query_time}",
  "query": "{query}",
  "gold_answer": "{gold_answer}",
  "referenced_reasoning_chain": "{referenced_reasoning_chain}",
  "source_messages": [
    {
      "role": "{role}",
      "content": "{source_message_content}",
      "timestamp": "{timestamp}"
    }
  ],
  "constructed_memory": "{compressed_memory}",
  "evaluation_rules": [
    "仅使用 source_messages 对 faithfulness 评分。评判 faithfulness 时不得使用 gold_answer 或
  referenced_reasoning_chain。",
    "每条源消息都可能包含 timestamp。在核查日期、时间先后关系、顺序和时间间隔时，将该时间戳视为源证据的一部
  分。",
    "将 query_time 视为提出查询的时间。在评判时间相关性、时效性、时间顺序和知识更新时，将其与源消息的时间戳
  进行比较。",
    "faithfulness 检查 constructed_memory 相对于 source_messages（包括其中的时间戳）是否包含无依据、
  捏造、矛盾或过度推断的信息。",
    "结合 query、gold_answer 和 referenced_reasoning_chain 对 utility 评分。",
    "utility 检查 constructed_memory 是否有助于回答 query、得出或验证 gold_answer，或完成
  referenced_reasoning_chain 中的推理。",
    "utility 不要求直接的单跳贡献。如果间接有用的证据、约束、背景、消歧信息、时间顺序、实体或上下文有助于最
  终答案或推理链，也可以获得 utility 分数。",
    "独立评判 faithfulness 和 utility；一条忠实但无用的记忆可以是 faithfulness=2、utility=0。",
    "一条有用但不忠实的记忆必须为 faithfulness=0；不得因其与 gold_answer 或
  referenced_reasoning_chain 一致而提高 faithfulness。",
    "如果 constructed_memory 只是通用背景，或重复了与 gold_answer 和 referenced_reasoning_chain 无
  关的信息，则应给予较低的 utility 分数。",
    "如果 constructed_memory 保留了 gold_answer 或 referenced_reasoning_chain 所需的关键直接或间接
  证据，则应给予较高的 utility 分数。",
    "不确定时，选择较低的分数。"
  ],
  "rubric": {
    "faithfulness": {
      "0": "包含 source_messages 无法支持的信息、捏造内容、明显矛盾，或将不确定信息表述为事实。",
      "1": "基本忠实，但存在轻微歧义、轻度过度概括、措辞不精确，或包含 source_messages 未明确支持的细节。
  ",
      "2": "完全忠实；所有关键信息都得到 source_messages 的明确支持，不存在矛盾、幻觉或过度推断。"
    },
    "utility": {
      "0": "对 gold_answer 或 referenced_reasoning_chain 没有帮助、与之无关，或会误导最终答案或推
  理。",
      "1": "对 gold_answer 或 referenced_reasoning_chain 有一定帮助，包括间接支持、有用背景、消歧上
  下文、时间顺序、实体、约束或部分相关证据。",
      "2": "对 gold_answer 或 referenced_reasoning_chain 有很强帮助；保留了得出、验证或解释答案所需
  的关键直接或间接证据、约束、事实或上下文。"
    }
  },
  "required_output": {
    "faithfulness": 0,
    "utility": 0,
    "unsupported_claims": [],
    "useful_evidence": [],
    "reason": "简短的中文理由"
  }
```

```
}
```

Listing 2: 记忆质量评审器的系统消息、评分标准与输出模式。

G.3 答案生成提示

代码清单 3 给出了共享的答案生成提示。压缩记忆条件与金标/文本记忆条件之间的差异仅在于记忆块的描述和呈现方式。

```
SYSTEM PROMPT

请根据所提供的记忆，简洁、忠实地回答用户问题。


USER PROMPT TEMPLATE FOR COMPRESSED MEMORY

你正在使用经过压缩、按时间顺序排列的对话记忆回答用户问题。
在记忆相关时使用它。如果记忆不足，请说明缺少哪些信息，而不要捏造事实。

# 查询
查询时间：{query_time}
查询：{query}

# 记忆
[{timestamp}] {role}: {compressed_content}
...

# 回答


USER PROMPT TEMPLATE FOR GOLD/TEXT MEMORY

你正在使用按时间顺序排列的对话记忆回答用户问题。
在记忆相关时使用它。如果记忆不足，请说明缺少哪些信息，而不要捏造事实。

# 查询
查询时间：{query_time}
查询：{query}

# 记忆
{memory_text}

# 回答
```

Listing 3: 用于答案生成的系统消息与用户消息模板。